Tacet:
An Honest Approach to Value Reification

Ramon Opazo
ramon.opazo.c@gmail.com
github.com/RamonOpazo/tacet-lang
Independent Research

August 18, 2026

Chapter 1

Introduction

Mainstream type systems provide weaker execution guarantees than their function types appear to suggest, and they leave that weakening implicit. A signature $f : A \to \tau$ claims that evaluation, given an input in A, produces a value of type τ. Yet in most languages evaluation may instead terminate through a panic, propagate an exception, or fail to terminate at all—and none of these possibilities is visible in the type. The declared type and the semantic contract it seems to establish diverge, silently.

The divergence can be made precise. Consider the semantic outcome space of a function,

$$[\![f]\!] : A \longrightarrow \tau + E + \{\bot\},$$

where τ ranges over successful values, E over exceptional termination, and $\bot$ over computations that yield no value at all, such as divergence or undefined evaluation. A mainstream type system typically exposes only

$$f : A \longrightarrow \tau,$$

suppressing E and $\bot$ entirely. The critical dishonesty is that the type describes only the successful, value-producing behaviour of a function, while the signature is read as though it described the whole of $[\![f]\!]$. The gap between the two is the subject of this work. We seek a calculus that keeps the declared type honest with respect to the semantic outcome space, while exposing only the minimal machinery required to close that gap.

1.1 Mainstreamn failure modeling

Existing disciplines for handling failure differ, in essence, over a single question: where is failure reified? Three answers are already in wide use, and the contrast between them motivates ours.

In the value type (monadic encapsulation). Languages such as Haskell may represent failure explicitly in the return type, transforming a function that might fail from $A \to B$ into $A \to \mathrm{Maybe}\, B$. The possibility of failure is thereby made static and propagated through subsequent computation. But the failure has been relocated into the value domain: $\mathrm{Maybe}\, B$ replaces the domain B with one that additionally represents absence,

so the caller must continuously reason about the computational context—the wrapper—alongside the underlying value. The resulting function is total with respect to its monadic domain, yet its value space has been weakened, because the absence of a value is now representable within the enclosing type.

Nowhere in the type (exceptions). Languages such as Python model failure as a runtime control-flow object that interrupts evaluation and propagates outward until handled. A function annotated $f : A \to B$ may return a value of type B, or it may raise, with no indication of that possibility in its type. The value space stays clean—a returned B really is a B—but honesty is lost: the signature asserts a totality the function does not possess, and the exceptional path remains an invisible control-flow escape.

These two disciplines sit at opposite corners of a trade-off. The monadic discipline buys honesty by weakening the value space; the exceptional discipline preserves the value space by abandoning honesty.

In the typing judgment (ours). We take a third position. Failure is reified neither inside the value type nor nowhere, but in the typing judgment itself, as a linear obligation recorded alongside—not inside—the value type. Judgments take the form

$$\Gamma;\ \Delta \vdash e : \tau,$$

where τ is the ordinary value type, kept clean, and Δ is a linear context of obligations the computation must discharge. The value type continues to promise exactly τ; the honesty is carried by Δ. This is the central move of the paper: we obtain a signature that is honest about failure without enriching τ, by placing the obligation in the context rather than in the value.

1.2 Strong and weak value spaces

The distinction just drawn can be stated once and for all. A value space is *weak* when computational failure is representable within the same domain as successful values: $\mathrm{Maybe}\, B$ is weak because the absence of a B inhabits the very type that is supposed to contain values of B. A value space is *strong* when it preserves the separation between values and computational outcomes: τ contains only values of type τ, while failure, divergence, and exceptional termination remain distinct computational outcomes, tracked elsewhere.

Tacet maintains a strong value space. No element of τ ever denotes a failure, and $\bot$ is never an inhabitant of a value type. Whatever can go wrong in a computation is a computational outcome accounted for outside τ—and, as the next section explains, accounted for explicitly.

1.3 Honesty is disclosure, not the absence of failure

We do not attempt to abolish failure, and we cannot abolish divergence—the halting problem forbids deciding, in general, whether an arbitrary computation terminates. Our principle is weaker and attainable: every way in which a computation may deviate from returning a value of its declared type must be disclosed. Honesty, in Tacet, is not the absence of $\bot$ but its disclosure.

The mechanism rests on a three-level separation between values, witnesses, and obligations.

Ordinary evaluation produces a value—an element of the value domain V. When evaluation instead encounters a computational phenomenon that is not an ordinary value—a possible failure, an unbounded recursion, an asynchronous wait, an intrinsically infinite loop—it produces a witness. A witness is a stateful object: it is raised from inside the computation and records the phenomenon that occurred, including the type of the value that was expected. A witness may in turn expose a linear obligation: a nominal signal, carrying no payload beyond its own identity, whose message is simply that a computation has left the value space and must be brought back into it.

The 'tacet' construct is the syntactic bridge that lets a computation raise such a witness deliberately, converting a distinguished class of tags into linear witnesses. Once an obligation is exposed, it must be discharged. Discharge takes one of two forms: an obligation may be reified back into an ordinary value, or it may be deferred, consuming the current obligation and producing a successor that some later step must in turn resolve. Linearity governs the whole process: an obligation cannot be duplicated and cannot be silently dropped. Its only alternative to being discharged is an explicit, terminal failure—surfaced loudly, never absorbed into a quiet default or a silent bottom.

Some phenomena resist discharge by their nature. A genuinely unbounded loop cannot be reified at a finite point, and a fire-and-forget effect has, by design, no value to recover. Such phenomena may be renounced rather than discharged—but only by declaring a corresponding capability in the function's signature. Declaring the capability to diverge, for instance, readmits $\bot$ into that function's outcome space; crucially, it does so visibly, disclosing the risk to every caller. A function that declares no such capability is guaranteed, by construction, to return a value of its declared type. In this way the full outcome space $\tau + E + \{\bot\}$ is either collapsed onto τ or expanded exactly as far as the signature announces, and no further. Whatever a function can do beyond returning τ is precisely what its type and its capabilities disclose.

1.4 Contributions

This paper develops the calculus sketched above and establishes its basic metatheory. Concretely, it contributes:

- a computational model that separates values from obligations, sustaining a strong value space under honest signatures with minimal syntactic surface;
- a uniform treatment in which failure, divergence, asynchrony, and looping are all witnessed computational outcomes exposing linear obligations, organized by a small algebra of admissible dispositions whose structure constrains which obligations may co-occur;
- linear discharge rules—reification and deferral—together with a type-preserving/type-transforming distinction that renders a branch's codomain discipline statically analyzable; and

- a metatheory: preservation and progress for the linear fragment, a conservation result stating that no obligation is ever silently dropped, and an honesty theorem establishing that the outcomes of a well-typed program coincide with its declared type, up to the capabilities its signature explicitly declares.